1. (2 Punkte) Sortiere die Zahlenfolge mit SelectionSort. Schreibe für die ersten drei Durchgänge je eine Zeile.

   `13 4 92 42 11 7 12`

Lösung:

```
4 13 92 42 11 7 12
4 7 92 42 11 13 12
4 7 11 42 92 13 12
```

2. (4 Punkte) Notiere den Code, der für den SelectionSort-Algorithmus fehlt. Gib auch die Stufe der Einrückung an.

```
def selection_sort(a):                  #E0
                                      ???(1)
        best = i                        #E2
        best_val = a[i]                 #E2
        for j in range(i+1,len(a)): #E2
                                      ???(2)
            best= j                     #E4
            best_val = a[j]             #E4
        a[best],a[i] = a[i],a[best]    #E2
```

Lösung:

```
(1) for i in range(len(a)-1):     #E1
(2) if a[j] < best_val:           #E3
```

3. (3 Punkte) In unserer Implementierung des SelectionSort-Algorithmus wurde ein print-Statement eingefügt. Welche Zahlenfolge wird bei dem angegebenen Aufruf von SelectionSort ausgegeben? (Der print-Parameter end=' ' sorgt dafür, dass die Zahlen nicht in einer neuen Zeile, sondern mit blank getrennt ausgegeben werden.)

```
def selection_sort(a):
    for i in range(len(a)-1):
        best = i
        best_val = a[i]
        for j in range(i+1,len(a)):
            if a[j] < best_val:
                best = j
                print(best,end= ' ')
                best_val = a[j]
        a[best],a[i] = a[i],a[best]

selection_sort([7,3,1,8,5,12,5,4])
```

Lösung:

```
1 2 4 7 4 6 6 7
```

4. (2 Punkte) Unsere Implementierung des SelectionSort-Algorithmus wurde mit anderen Variablennamen neu aufgeschrieben. Dabei haben sich Fehler eingeschlichen. Korrigiere die fehlerhaften Stellen.

```
def selection_sort(hugo):
    for kai in range(len(hugo)):
        otto = kai
        lena = hugo[kai]
        for j in range(kai+1,len(hugo)):
            otto = j
            lena = hugo[j]
        hugo[otto],hugo[kai] = hugo[kai],hugo[otto]
```

Lösung:

```
In Zeile 2: for kai in range(len(hugo)-1):
Nach Zeile 5 fehlt eine Zeile:
if hugo[j] < lena:
die beiden nächsten Zeilen müssen eingerückt werden,
so dass sie zu diesem if gehören.
```

5. (2 Punkte) Sortiere die Zahlenfolge mit BubbleSort. Schreibe für die ersten drei Durchgänge je eine Zeile.

```
2 22 14 25 1 13 9
```

Lösung:

```
2 14 22 1 13 9 25
2 14 1 13 9 22 25
2 1 13 9 14 22 25
```

6. (3 Punkte) In unserer Implementierung des Bubblesort-Algorithmus wurde ein print-Statement eingefügt. Welche Zahlenfolge wird bei dem angegebenen Aufruf von Bubblesort ausgegeben? (Der print-Parameter end=' ' sorgt dafür, dass die Zahlen nicht in einer neuen Zeile, sondern mit blank getrennt ausgegeben werden.)

```
def bubble_sort(a):
    getauscht = True
    while getauscht:
        getauscht = False
        for i in range(len(a)-1):
            if a[i] > a[i+1]:
                a[i],a[i+1]=a[i+1],a[i]
                print(a[i],end=' ')
                getauscht = True

bubble_sort([7,4,2,3,1])
```

Lösung:

```
4 2 3 1 2 3 1 1 1
```

7. (2 Punkte) In unserer Implementierung des Bubblesort-Algorithmus wurden Variablennamen geändert. Dabei haben sich Fehler eingeschlichen. Gib die Zeilennummern mit den Fehlern an und korrigiere die fehlerhaften Stellen. (Wir zählen Zeilennummern mit 1 beginnend).

```
def bubble_sort(alpha):
    beta = True
    while beta:
        beta = False
        for delta in range(len(alpha)-1):
            if alpha[gamma] > alpha[gamma+1]:
                alpha[gamma], alpha[gamma+1]=alpha[gamma+1], alpha[gamma]
            beta = True
```

Lösung:

```
In Zeile 5: for gamma in range(len(alpha)-1):
In Zeile 8: beta = True  muss weiter eingerückt werden (wie Zeile 7)
```

8. (2 Punkte) Sortiere die Zahlenfolge mit InsertionSort. Schreibe für die ersten drei Durchgänge je eine Zeile.

`12 5 9 3 15 7 1`

Lösung:

```
5 12 9 3 15 7 1
5 9 12 3 15 7 1
3 5 9 12 15 7 1
```

9. (5 Punkte) Die Liste `a = [24, 4, 17, 88, 42, 12, 7]` wird mit dem rekursiven mergeSort-Algorithmus aus dem Unterricht sortiert.
   a. Wieviel mal wird `merge` aufgerufen?
   b. Wieviel mal wird `mergeSort` aufgerufen? (der erste Aufruf zählt mit).

Lösung: a. 6 b. 13

Bei jedem Aufruf von `merge` wird eine Liste zurückgegeben. Notiere die Listen in der Reihenfolge, in der sie zurückgegeben werden.

Lösung:

```
4 17
4 17 24
42 88
7 12
7 12 42 88
4 7 12 17 24 42 88
```

10. (2 Punkte) Welche Komplexität hat die Laufzeit von BubbleSort im best, worst und average-case? In welcher Komplexitätsklasse ist der zusätzliche Platzbedarf?

Lösung:
best: $O(n)$, average: $O(n^2)$, worst: $O(n^2)$, Platz: $O(1)$

11. (2 Punkte) Welche Komplexität hat die Laufzeit von InsertionSort im best, worst und average-case? In welcher Komplexitätsklasse ist der zusätzliche Platzbedarf?

Lösung:
best: $O(n)$, average: $O(n^2)$, worst: $O(n^2)$, Platz: $O(1)$

12. (2 Punkte) Welche Komplexität hat die Laufzeit von MergeSort im best, worst und average-case? In welcher Komplexitätsklasse ist der zusätzliche Platzbedarf?

Lösung:
best: $O(n \cdot \log n)$, average: $O(n \cdot \log n)$, worst: $O(n \cdot \log n)$, Platz: $O(n)$

13. (4 Punkte) Vergleiche BubbleSort und MergeSort. Gib für beide die Laufzeit im worst case, den zusätzlichen Platzbedarf und die Stabilität an. Welches Verfahren nehmen wir für eine Liste mit 1 Million Elementen?

Lösung: BubbleSort: worst case $O(n^2)$, Platz $O(1)$, stabil.
MergeSort: worst case $O(n \cdot \log n)$, Platz $O(n)$, stabil.
Bei 1 Million Elementen nehmen wir MergeSort, denn n^2 wäre viel zu groß. Dafür brauchen wir zusätzlichen Speicher.

14. (3 Punkte) Die Liste ist alphabetisch nach Namen geordnet: Anna 3, Ben 2, Cem 3, Dora 1, Emil 2. Sie wird mit einem stabilen Verfahren nach der Note sortiert.
    (a) Gib die sortierte Liste an.
    (b) Was bedeutet stabil?

Lösung: a) Dora 1, Ben 2, Emil 2, Anna 3, Cem 3
b) Elemente mit gleichem Sortierschlüssel behalten ihre bisherige Reihenfolge.

15. (3 Punkte) Die Liste `5a 5b 2` wird mit SelectionSort sortiert. Die Buchstaben zeigen nur, um welche 5 es sich handelt.
    (a) Wie sieht die Liste danach aus?
    (b) Ist SelectionSort stabil? Begründe.

Lösung: a) `2 5b 5a`
b) Nein. Beim Tauschen der 2 mit `5a` springt `5a` über `5b` hinweg, die Reihenfolge der gleichen Elemente ändert sich.

16. (4 Punkte) Vergleiche BubbleSort und MergeSort. Gib für beide die Laufzeit im worst case, den zusätzlichen Platzbedarf und die Stabilität an. Welches Verfahren nehmen wir für eine Liste mit 1 Million Elementen?

Lösung: BubbleSort: worst case $O(n^2)$, Platz $O(1)$, stabil.
MergeSort: worst case $O(n \cdot \log n)$, Platz $O(n)$, stabil.
Bei 1 Million Elementen nehmen wir MergeSort, denn n^2 wäre viel zu groß. Dafür brauchen wir zusätzlichen Speicher.